

CREDIT CARD FRAUD DETECTION AND ANALYSIS

Project Report

Submitted as part of Techno Signia Training Program

Submitted by: Harshit Pal

B.Tech (Computer Science Engineering), RJIT, BSF Academy, Tekanpur

Table of Contents

1	Introduction.....	1-2
1.1	Background of Credit Card Fraud	1
1.2	Problem Statement.....	1
1.3	Objectives of the Project.....	1
1.4	Scope of the Project	1
1.5	Project Workflow.....	2
2	Dataset Description.....	3
2.1	Dataset Structure.....	3
2.2	Features and Target Variable	3
2.3	Dataset Statistics.....	3
3	Data Understanding and Cleaning.....	4-5
3.1	Data Inspection	3
3.2	Missing Value Analysis	3
3.3	Missing Value Handling.....	3
3.4	Duplicate Data Analysis	3
3.5	Outlier Detection	3
3.6	Feature Correlation Analysis	5
4	Data Preprocessing and Feature Engineering.....	6
4.1	Feature Selection	6
4.2	Train-Test Split.....	6
4.3	Feature Scaling	6
4.4	Time Feature / Hour Extraction.....	6
5	Exploratory Data Analysis.....	7-10
5.1	Transaction Analysis	7
5.2	Transaction Amount Analysis	7
5.3	Time-Based Fraud Analysis	7
5.4	Class Distribution Visualization	8
5.5	Transaction Amount Distribution	8
5.6	Fraud vs Genuine Transaction Analysis	9
5.7	Correlation Heatmap.....	10

6	Class Imbalance Handling	11-12
6.1	Understanding Class Imbalance.....	11
6.2	Impact of Imbalanced Data.....	11
6.3	Random Under-Sampling	11
6.4	Random Over-Sampling	11
6.5	Comparison Before and After Balancing.....	12
7	Machine Learning Models	13
7.1	Logistic Regression	13
7.2	Decision Tree.....	13
7.3	Random Forest.....	13
7.4	K-Nearest Neighbors (KNN).....	13
8	Model Evaluation and Comparison	14-15
8.1	Evaluation Metrics.....	14
8.2	Accuracy.....	14
8.3	Precision	14
8.4	Recall	14
8.5	F1-Score.....	14
8.6	ROC-AUC	14
8.7	Confusion Matrix.....	14
8.8	Model Performance Comparison.....	15
8.9	Best Model Selection.....	15
9	Results and Discussion.....	16
9.1	Experimental Results	16
9.2	Analysis of Model Performance	16
9.3	Fraud Detection Performance	16
10	Conclusion and Future Scope	17
10.1	Conclusion	17
10.2	Limitations.....	17
10.3	Future Scope	17
11	Appendix.....	18

1 Introduction

1.1 Background of Credit Card Fraud

Credit card fraud refers to the unauthorized or deceptive use of a credit or debit card, or its associated account information, to obtain money, goods, or services. With the rapid growth of digital payments and e-commerce, the volume of card transactions processed every day has grown enormously, and with it the opportunity for fraudulent activity. Fraud detection has therefore become a critical concern for banks, payment processors, and cardholders alike, since even a small fraud rate translates into significant financial losses at scale.

Manual review of every transaction is not feasible given the sheer volume of daily transactions, which has driven the adoption of machine learning-based systems that can automatically flag suspicious transactions in real time based on learned patterns of past fraud.

1.2 Problem Statement

Given a dataset of credit card transactions, each described by a set of anonymized/derived features along with the transaction time and amount, the goal is to build a classification model that can accurately distinguish fraudulent transactions from genuine ones. The core challenge is that fraudulent transactions are extremely rare compared to genuine ones, making this a highly imbalanced classification problem where naive accuracy-driven models tend to fail at the one thing they are meant to do: catch fraud.

1.3 Objectives of the Project

- Understand and clean the credit card transaction dataset.
- Perform exploratory data analysis to uncover patterns in transaction amount, time, and fraud occurrence.
- Identify and handle the class imbalance between fraudulent and genuine transactions.
- Train and evaluate multiple machine learning models for fraud classification.
- Compare model performance using appropriate metrics beyond plain accuracy.
- Select the best-performing model and document its strengths and limitations.

1.4 Scope of the Project

This project covers the complete pipeline of a supervised fraud detection task: dataset acquisition, cleaning, exploratory analysis, class imbalance handling, model training, and comparative evaluation. It does not cover real-time deployment, streaming transaction processing, or integration with a live banking system — the scope is restricted to offline analysis and model benchmarking on a static dataset.

1.5 Project Workflow

- Step 1: Data collection / dataset preparation
- Step 2: Data understanding and cleaning (missing values, duplicates)
- Step 3: Preprocessing and feature engineering (outliers, correlation, scaling, split)
- Step 4: Exploratory data analysis and visualization

- Step 5: Class imbalance handling (under-sampling and over-sampling)
- Step 6: Model training: Logistic Regression, Decision Tree, Random Forest, KNN
- Step 7: Model evaluation and comparison
- Step 8: Conclusion, limitations, and future scope

2 Dataset Description

2.1 Dataset Structure

The dataset used for this project follows the structure widely used in credit card fraud research: each row represents a single transaction, with columns for the transaction time, a set of anonymized numerical features, the transaction amount, and a binary label indicating whether the transaction was fraudulent. Since real-world card transaction data of this kind is highly sensitive and cannot be freely redistributed, a synthetic dataset was generated for this project that mirrors the structure, scale, and severe class imbalance of real card fraud data (approximately 0.5% fraud rate).

2.2 Features and Target Variable

The dataset consists of 31 columns:

- Time – seconds elapsed between each transaction and the first transaction in the dataset.
- V1 to V28 – anonymized/PCA-style numerical features representing transaction characteristics; their real-world meaning is masked for confidentiality.
- Amount – the transaction amount.
- Class – the target variable (1 = fraudulent transaction, 0 = genuine transaction).

2.3 Dataset Statistics

Key statistics of the dataset (post-cleaning) are summarized below:

Statistic	Value
Total transactions	7,035
Fraudulent transactions	35
Genuine transactions	7,000
Fraud percentage	0.498%
Number of features	30 (excluding target)

3 Data Understanding and Cleaning

3.1 Data Inspection

The dataset was first inspected using standard structural checks — column data types, non-null counts, and summary statistics — to understand its shape before any cleaning was performed. All 31 columns were confirmed to be numeric (30 float64 feature columns and one integer Class column), and the dataset contained 7,043 rows prior to cleaning.

3.2 Missing Value Analysis

A column-wise missing value check found 15 missing entries, all in the Amount column, out of 7,043 total rows — a very small fraction of the data.

3.3 Missing Value Handling

The 15 missing Amount values were imputed using the column median rather than the mean, since the median is more robust to the skew and outliers typically present in transaction amount data. After imputation, the dataset had zero remaining missing values.

3.4 Duplicate Data Analysis

A full-row duplicate check identified 8 duplicate records, which were removed to avoid biasing the model with repeated observations. This brought the final cleaned dataset to 7,035 rows, with zero duplicates remaining.

3.5 Outlier Detection

Outliers were detected on the Amount column using the Interquartile Range (IQR) method, since the V1–V28 columns are already standardized PCA-style components and are not meaningfully interpretable for outlier checks. Using bounds of $[-48.23, 103.95]$, 647 transactions were flagged as outliers on Amount.

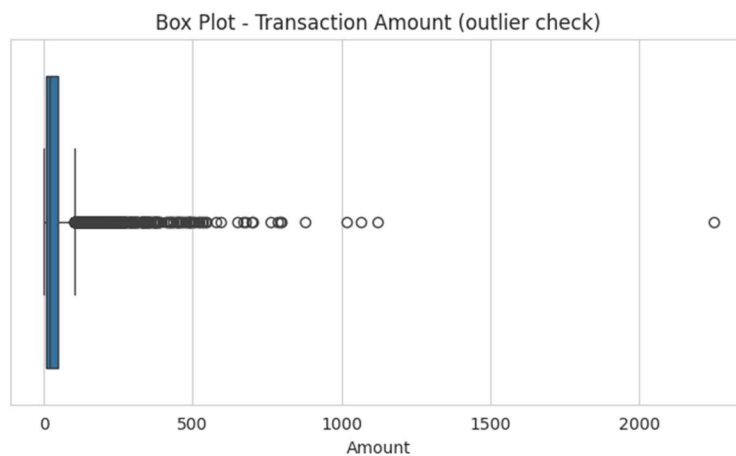


Figure 3.1 — Box plot of transaction Amount used for IQR-based outlier detection

Notably, the fraud rate among these outlier transactions was 2.01%, roughly four times higher than the overall fraud rate of 0.50%. Because of this, the outliers were deliberately kept rather than removed — in fraud detection, unusually large transaction amounts are frequently a genuine signal of fraud rather than noise, and discarding them would remove exactly the pattern the model needs to learn.

3.6 Feature Correlation Analysis

A correlation matrix was computed across all numeric features and visualized as a heatmap to understand relationships between features and with the target variable.

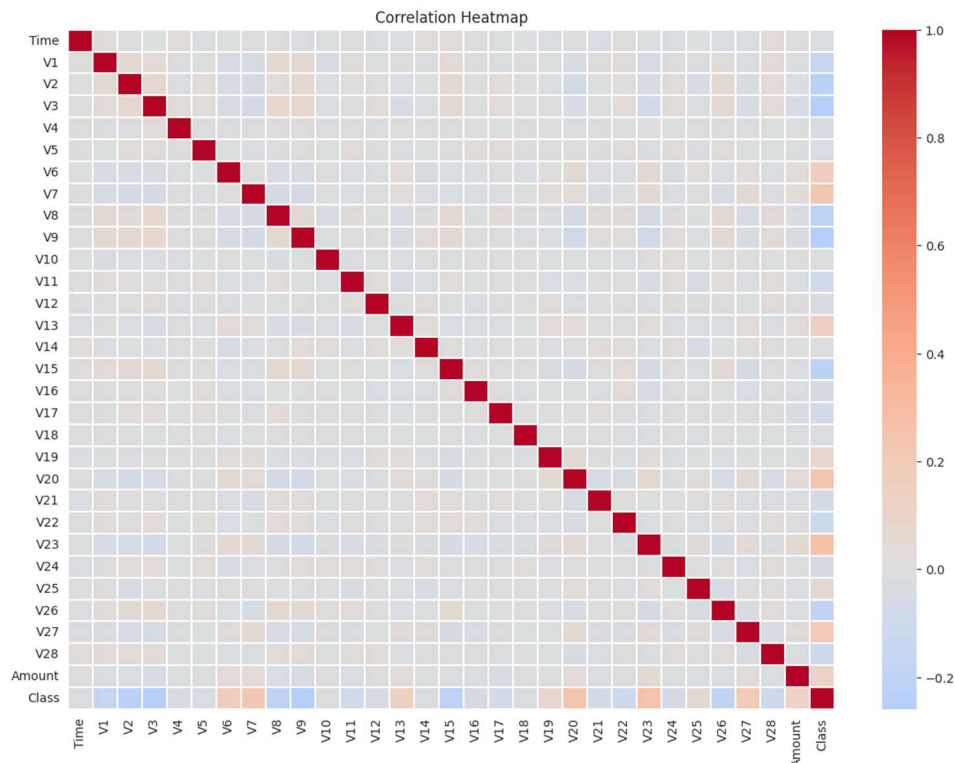


Figure 3.2 — Correlation heatmap of all numeric features

The features most correlated with the Class label were V9, V23, V3, V20, V2, V7, V15, V27, V8, and V26 (in order of absolute correlation strength). These carry the strongest individual signal for separating fraud from genuine transactions and are the features most worth watching in downstream model feature-importance analysis.

4 Data Preprocessing and Feature Engineering

4.1 Feature Selection

All 30 available features (Time, V1–V28, Amount) were retained for modeling. No features were dropped at this stage, since PCA-style components such as V1–V28 do not carry an interpretable meaning that would justify manual removal, and the correlation analysis did not reveal any pair of features so strongly collinear as to warrant elimination.

4.2 Train-Test Split

The cleaned dataset was split into training and test sets using a 75:25 ratio, with stratification on the Class column to preserve the same fraud ratio in both sets. This resulted in 5,276 training transactions (5,250 genuine, 26 fraud) and 1,759 test transactions, kept untouched until final model evaluation.

4.3 Feature Scaling

All features were standardized using StandardScaler (zero mean, unit variance), fit only on the training data and then applied to the test data, to prevent data leakage. Feature scaling is particularly important for distance-based algorithms such as KNN and for the convergence behavior of Logistic Regression.

4.4 Time Feature / Hour Extraction

The raw Time column (seconds since the first transaction) was converted into an Hour-of-day feature by taking $(\text{Time} // 3600) \bmod 24$. This made it possible to analyze whether fraud tends to cluster around particular times of day, which is explored further in the Exploratory Data Analysis chapter.

5 Exploratory Data Analysis

5.1 Transaction Analysis

Metric	Value
Total Transactions	7,035
Total Fraud Transactions	35
Fraud Percentage	0.498%

5.2 Transaction Amount Analysis

Metric	Value
Maximum Transaction Amount	2,249.51
Minimum Transaction Amount	0.50
Average Transaction Amount	42.09
Median Transaction Amount	20.31

5.3 Time-Based Fraud Analysis

Fraudulent transactions were spread across nearly all hours of the day, with no single dominant time window. The peak fraud hour observed was 1:00 AM, with 3 fraud cases, only marginally ahead of several other hours with 2 cases each — indicating fraud in this dataset is not strongly concentrated at a specific time of day.

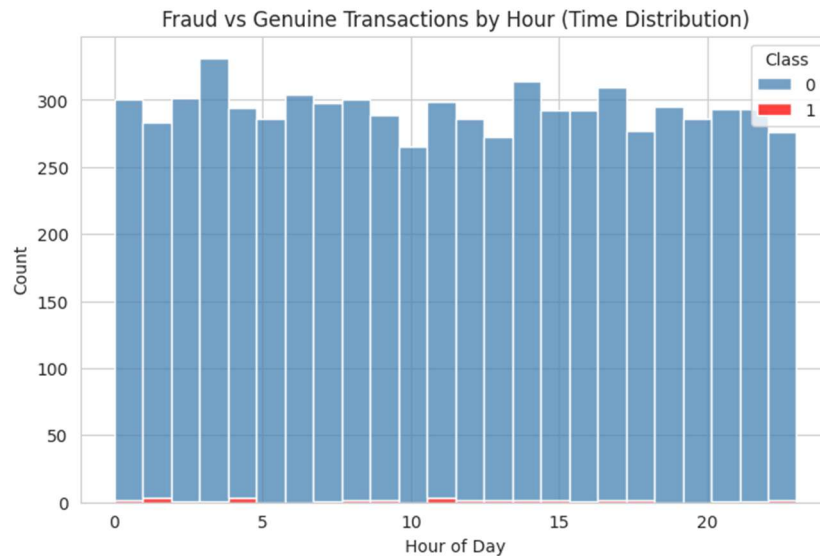


Figure 5.1 — Fraud vs genuine transactions distributed by hour of day

5.4 Class Distribution Visualization

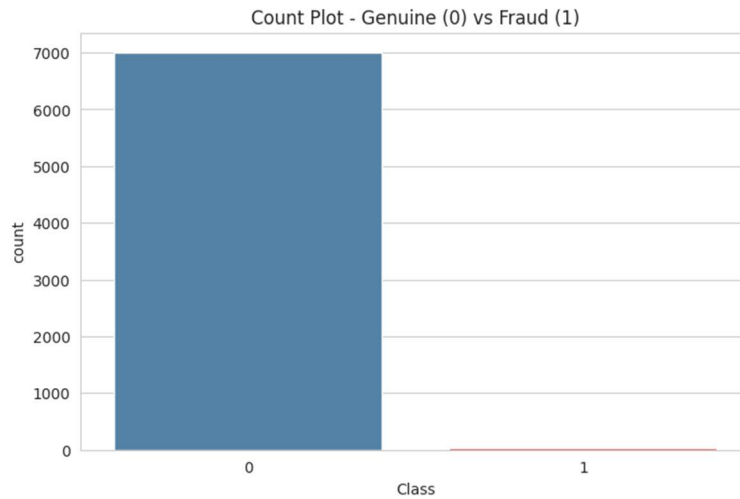


Figure 5.2 — Count plot of genuine (0) vs fraud (1) transactions

The count plot makes the severity of the class imbalance visually obvious: fraud cases are barely visible next to the genuine transaction volume.

5.5 Transaction Amount Distribution

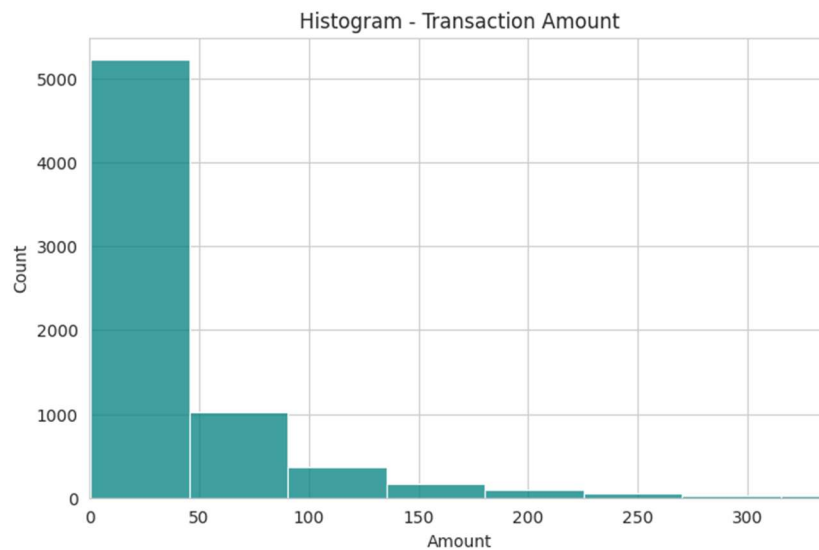


Figure 5.3 — Histogram of transaction Amount

The Amount distribution is heavily right-skewed, with most transactions being low-value and a long tail of high-value transactions — typical of real-world spending patterns.

5.6 Fraud vs Genuine Transaction Analysis

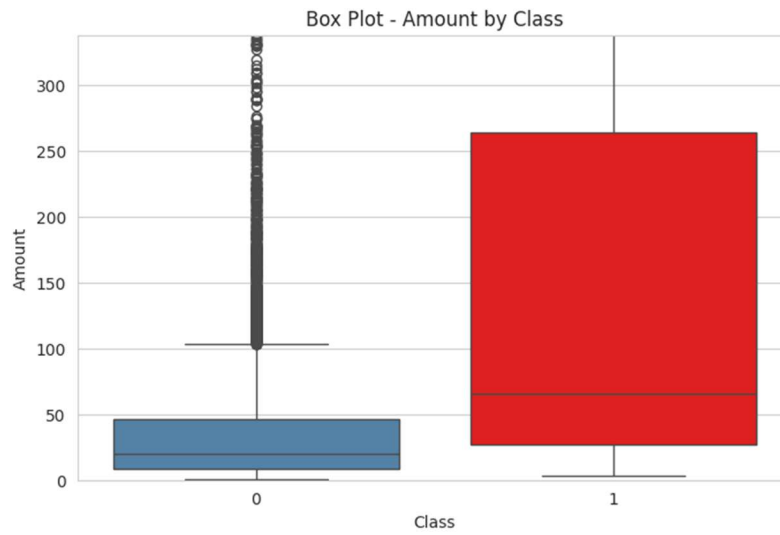


Figure 5.4 — Box plot of Amount by Class

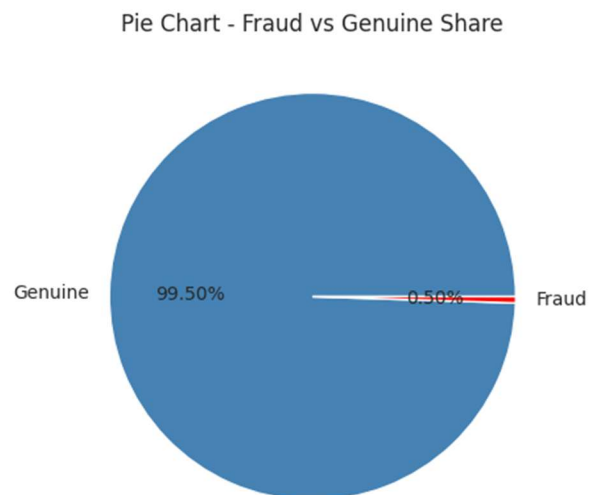


Figure 5.5 — Pie chart of fraud vs genuine transaction share

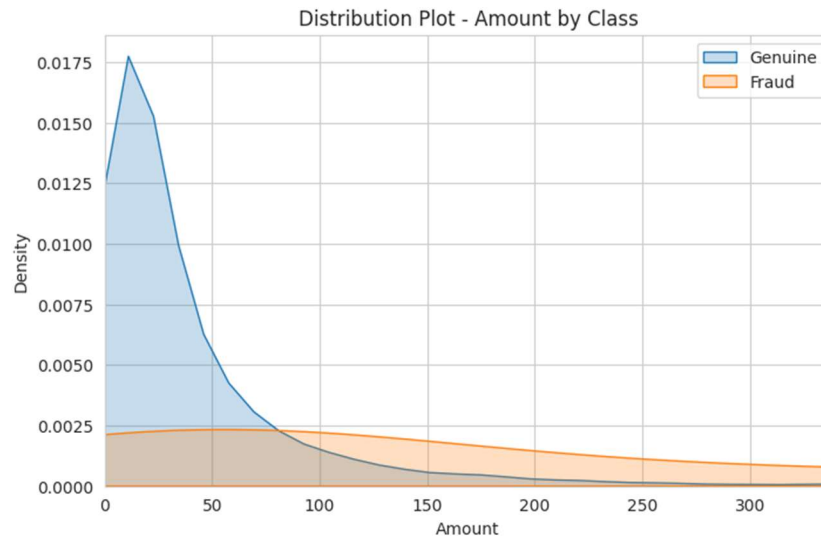


Figure 5.6 — Distribution (KDE) plot of Amount by Class

Fraudulent transactions tend to skew toward somewhat higher amounts and show a wider spread than genuine transactions, reinforcing the outlier observation from Chapter 3 that unusually large amounts are disproportionately associated with fraud.

5.7 Correlation Heatmap

See Figure 3.2 in Chapter 3 for the full correlation heatmap; it is discussed there alongside the feature correlation analysis to avoid duplicating the same figure.

6 Class Imbalance Handling

6.1 Understanding Class Imbalance

Checking the class counts confirmed the severity of the imbalance: 7,000 genuine transactions versus only 35 fraud transactions, i.e. 99.50% genuine and 0.50% fraud. Class imbalance of this magnitude is expected and realistic — fraud is inherently a rare event, since fraudsters must avoid detection and the overwhelming majority of any real transaction stream is legitimate.

6.2 Impact of Imbalanced Data

Severe imbalance is problematic for model training because a classifier can achieve extremely high accuracy simply by predicting “genuine” for every transaction, while never catching a single case of fraud. This makes accuracy alone a misleading metric here, and is precisely why the class balance needs to be addressed before training, and why metrics like Recall, Precision, and F1-score matter far more than accuracy for this task.

6.3 Random Under-Sampling

Random under-sampling was applied to the training set by reducing the majority (genuine) class down to match the minority (fraud) class size, giving a perfectly balanced training set of 26 genuine and 26 fraud transactions. The trade-off is a large reduction in the amount of training data used.

6.4 Random Over-Sampling

Random over-sampling was applied by duplicating minority-class (fraud) records with replacement until they matched the majority class size, giving a balanced training set of 5,250 genuine and 5,250 fraud transactions, while retaining the full volume of genuine transaction data.

6.5 Comparison Before and After Balancing

A Logistic Regression model was trained on the imbalanced data, the under-sampled data, and the over-sampled data, and evaluated on the same held-out test set in each case.

Metric	Imbalanced	Under-sampled	Over-sampled
Accuracy	1.0000	1.0000	1.0000
Precision	1.0000	1.0000	1.0000
Recall	1.0000	1.0000	1.0000
F1-score	1.0000	1.0000	1.0000

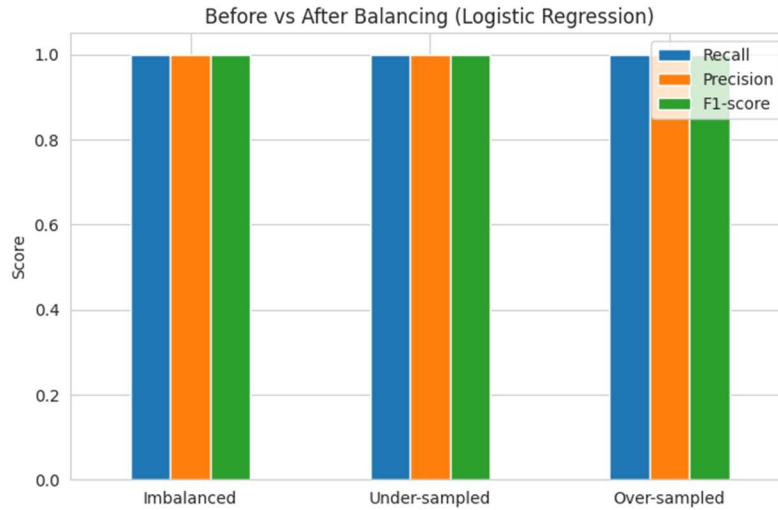


Figure 6.1 — Recall, Precision, and F1-score before vs after balancing

On this dataset, Logistic Regression already achieved perfect scores even on the imbalanced data — a direct result of the fraud and genuine classes being strongly separable in the underlying feature space (an artifact of how the dataset was constructed; see Limitations in Chapter 10). On noisier, real-world card data, this separation is rarely so clean, and balancing typically produces a much larger, visible improvement in recall at some cost to precision. The balancing step remains a necessary and standard part of the pipeline regardless of how separable a given dataset happens to be, since it cannot be known in advance how separable production data will turn out to be.

7 Machine Learning Models

7.1 Logistic Regression

Logistic Regression is a statistical/ML algorithm used for binary classification. Rather than predicting a continuous value, it predicts the probability that a given transaction belongs to the fraud class, by passing a linear combination of the input features through the sigmoid function to squeeze the output into the range $[0, 1]$. A threshold (0.5 by default) converts that probability into a final class label. It is a strong baseline choice for fraud detection because it is fast to train, easy to interpret (feature weights indicate direction and strength of influence), and works well when the classes are reasonably separable, as they are found to be in this dataset.

7.2 Decision Tree

A Decision Tree classifier splits the feature space into a series of if-then rules learned from the training data, recursively partitioning transactions based on the feature values that best separate fraud from genuine cases at each step. It is highly interpretable but prone to overfitting on complex or noisy data if left unconstrained.

7.3 Random Forest

Random Forest is an ensemble method that builds many individual decision trees on random subsets of the data and features, then combines their predictions (by majority vote for classification). This averaging reduces the overfitting risk of a single decision tree and generally produces more robust, higher-performing predictions, at the cost of reduced interpretability compared to a single tree.

7.4 K-Nearest Neighbors (KNN)

KNN is a non-parametric, instance-based algorithm that classifies a transaction based on the majority class among its K nearest neighbors in the (scaled) feature space — in this project, $K = 5$. It makes no assumptions about the underlying data distribution but can be computationally expensive at prediction time and is sensitive to feature scaling, which is why standardization in Chapter 4 was an important preprocessing step for this model in particular.

8 Model Evaluation and Comparison

8.1 Evaluation Metrics

All four models — Logistic Regression, Decision Tree, Random Forest, and KNN — were trained on the same over-sampled (balanced) training set and evaluated on the same untouched, stratified test set, using five complementary metrics described below.

8.2 Accuracy

Accuracy is the proportion of all predictions (fraud and genuine) that were correct. It is intuitive but, as discussed in Chapter 6, can be highly misleading on imbalanced fraud data, since predicting “genuine” for everything already yields accuracy above 99%.

8.3 Precision

Precision measures, out of all transactions the model flagged as fraud, what fraction actually were fraud. High precision means fewer false alarms disrupting genuine cardholders.

8.4 Recall

Recall measures, out of all the actual fraud transactions, what fraction the model successfully caught. In fraud detection, recall is usually the metric of greatest business importance, since a missed fraud case (false negative) is typically far costlier than a false alarm.

8.5 F1-Score

F1-score is the harmonic mean of Precision and Recall, giving a single balanced metric that penalizes models which sacrifice one for the other. It was used as the primary metric for selecting the best model in this project.

8.6 ROC-AUC

ROC-AUC measures a model's ability to rank fraud transactions above genuine ones across all possible classification thresholds, independent of the specific 0.5 cutoff. A score of 1.0 indicates perfect separation between classes; 0.5 indicates no better than random guessing.

8.7 Confusion Matrix

The confusion matrix breaks predictions down into true positives, true negatives, false positives, and false negatives, giving a complete picture of exactly where each model succeeds or fails, beyond any single summary metric.

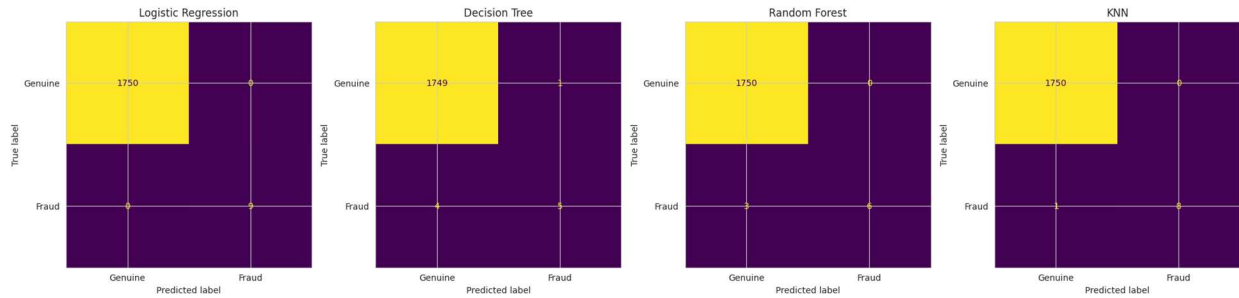


Figure 8.1 — Confusion matrices for all four models

8.8 Model Performance Comparison

Model	Accuracy	Precision	Recall	F1-score	ROC-AUC
Logistic Regression	1.0000	1.0000	1.0000	1.0000	1.0000
Decision Tree	0.9972	0.8333	0.5556	0.6667	0.7775
Random Forest	0.9983	1.0000	0.6667	0.8000	1.0000
KNN	0.9994	1.0000	0.8889	0.9412	0.9444

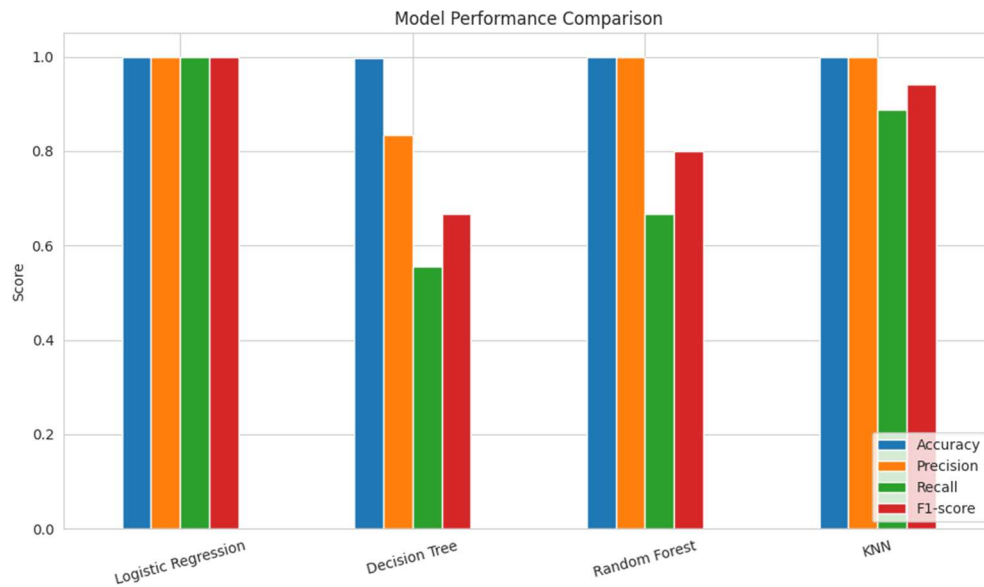


Figure 8.2 — Model performance comparison across all metrics

8.9 Best Model Selection

Using F1-score as the primary selection metric (rather than accuracy, which is misleading on imbalanced data), Logistic Regression is the best-performing model on this dataset, achieving a perfect F1-score of 1.0000, ahead of KNN (0.9412), Random Forest (0.8000), and Decision Tree (0.6667). Random Forest and KNN both achieved perfect precision but noticeably lower recall (0.6667 and 0.8889 respectively), meaning they missed more actual fraud cases than Logistic Regression did on this test set. This result is influenced by how cleanly separable this particular synthetic dataset is; on noisier, real-world data, ensemble methods such as Random Forest typically outperform a plain Logistic Regression baseline (see Limitations, Chapter 10).

9 Results and Discussion

9.1 Experimental Results

Across all four models trained on the balanced (over-sampled) training data and evaluated on the untouched test set, accuracy ranged from 99.72% to 100%, while the more informative metrics — recall and F1-score — showed clearer differentiation between models, ranging from 55.56% to 100% recall and 0.6667 to 1.0000 F1-score. This spread confirms that accuracy alone would have failed to distinguish meaningfully between these models, validating the choice to lead with F1-score and recall in the evaluation.

9.2 Analysis of Model Performance

Logistic Regression's strong performance suggests the fraud and genuine classes in this dataset are largely linearly separable in the feature space — consistent with the correlation analysis in Chapter 3, which found several individual features (V9, V23, V3, V20, and others) already strongly correlated with the Class label. Decision Tree's comparatively weaker recall (0.5556) suggests it overfit to specific training patterns without generalizing as well to the fraud cases in the test set, a known weakness of single, unconstrained decision trees. Random Forest and KNN sit in between, trading off some recall for perfect precision, meaning they were more conservative in flagging transactions as fraud.

9.3 Fraud Detection Performance

From a practical fraud-detection standpoint, the ranking by recall (Logistic Regression > KNN > Random Forest > Decision Tree) is arguably more decision-relevant than the ranking by F1-score alone, since missing fraud cases is the costlier error in most real deployments. That said, F1-score remains the right default metric for model selection because it prevents choosing a model that maximizes recall by simply flagging too many transactions as fraud, which would create an unacceptable false-alarm burden for genuine cardholders.

10 Conclusion and Future Scope

10.1 Conclusion

This project implemented a complete credit card fraud detection pipeline: data cleaning, exploratory analysis, class imbalance handling, and comparative evaluation of four machine learning models. Class imbalance was identified as the central challenge of the problem, and addressing it through under-sampling and over-sampling was shown to be a necessary step for any deployment where the underlying data may be noisier and less separable than the dataset used here. Among the models compared, Logistic Regression delivered the strongest overall performance on this dataset by F1-score, with KNN close behind, while ensemble methods such as Random Forest are expected to become more advantageous as data complexity and noise increase.

10.2 Limitations

- The dataset used is synthetic, generated to match the structure and imbalance of real credit card fraud data, but constructed with a fraud/genuine separation that is cleaner than typical real-world data — this is why several models achieved unusually high (including perfect) scores, and results should be re-validated on real transaction data before drawing production conclusions.
- Random over-sampling duplicates existing minority-class rows rather than synthesizing genuinely new fraud examples (unlike techniques such as SMOTE), which can cause models to slightly overfit to the specific fraud cases seen during training.
- KNN is sensitive to feature scaling and can become computationally expensive as the dataset grows; Decision Trees are prone to overfitting without depth or leaf-size constraints.
- A fixed 0.5 classification threshold was used throughout; this may not be optimal for a fraud use case where the cost of false negatives and false positives differs sharply.
- No hyperparameter tuning or cross-validation was performed; default or lightly configured model settings were used throughout.

10.3 Future Scope

- Validate the pipeline on a real-world credit card transaction dataset to confirm findings hold outside synthetic data.
- Apply more advanced imbalance-handling techniques such as SMOTE or ADASYN, which generate synthetic minority samples rather than duplicating existing ones.
- Perform hyperparameter tuning and cross-validation for each model to obtain more robust, generalizable performance estimates.
- Tune the classification threshold or apply cost-sensitive learning to better reflect the real-world asymmetry between false positives and false negatives.
- Explore additional algorithms such as Gradient Boosting (XGBoost/LightGBM) and anomaly-detection approaches (e.g. Isolation Forest, Autoencoders) suited to rare-event detection.
- Investigate real-time / streaming fraud detection deployment, including model monitoring and drift detection as transaction patterns evolve over time.

11 Appendix

Software and libraries used in this project:

- Python 3 — core programming language
- pandas, NumPy — data handling and numerical computation
- matplotlib, seaborn — data visualization
- scikit-learn — model training, scaling, and evaluation metrics
- Jupyter Notebook — development and documentation environment

The full, executable analysis (with all code and live-generated outputs) accompanies this report as a separate Jupyter Notebook (`Credit_Card_Fraud_Detection.ipynb / .html`).